

GCP Terraform + Ansible PostgreSQL Lab — Clear Steps (nothing pre-created)

Every step starts with mkdir/cd when a file is written. LABDIR is always \$HOME/tf-ansible-pg. If a command says No such file or directory, you skipped mkdir or you are not in LABDIR.

Do not open 5432 or 22 to 0.0.0.0. Do not run two terraform apply at once.

Step 0 — Create the lab folder and tools

```
mkdir -p "$HOME/tf-ansible-pg/env"
mkdir -p "$HOME/tf-ansible-pg/modules/network"
mkdir -p "$HOME/tf-ansible-pg/modules/iam"
mkdir -p "$HOME/tf-ansible-pg/modules/compute"
mkdir -p "$HOME/tf-ansible-pg/modules/sql"
mkdir -p "$HOME/tf-ansible-pg/ansible/playbooks"
mkdir -p "$HOME/tf-ansible-pg/ansible/inventories"
cd "$HOME/tf-ansible-pg"
pwd
ls -ld env modules ansible
gcloud --version | head -1
terraform version | head -1
ansible --version | head -1
```

If terraform is missing: install Terraform 1.6+ before Step 6. If ansible is missing: sudo apt-get install -y ansible.

Step 1 — Set project variables and write tfvars

```
cd "$HOME/tf-ansible-pg"
PROJECT_ID=$(gcloud config get-value project)
echo "PROJECT_ID=$PROJECT_ID"
test -n "$PROJECT_ID" && echo "project OK" || echo "STOP: run gcloud config set project YOUR_ID"
```

Next block writes env/terraform.tfvars. The env directory already exists from Step 0.

```
cd "$HOME/tf-ansible-pg"
PROJECT_ID=$(gcloud config get-value project)
cat > "$HOME/tf-ansible-pg/env/terraform.tfvars" <<EOF
project_id      = "${PROJECT_ID}"
region          = "asia-south1"
zone            = "asia-south1-a"
network_name    = "pg-lab-vpc"
subnet_name     = "pg-lab-subnet"
subnet_cidr     = "10.20.0.0/24"
vm_name         = "pg-ansible-vm"
sql_name        = "pg-lab-sql"
sql_password    = "ChangeMe-Lab-Only"
create_network  = false
EOF
cat "$HOME/tf-ansible-pg/env/terraform.tfvars"
```

project_id must be a real GCP id, not empty and not the characters \${PROJECT_ID}.

Step 2 — Enable APIs

```
gcloud services enable compute.googleapis.com sqladmin.googleapis.com iam.googleapis.com \
  servicenetworking.googleapis.com cloudresourcemanager.googleapis.com
```

Step 3 — Networking 1.1 VPC and subnet

If pg-lab-vpc already exists from an earlier lab, skip create and only run describe.

```
gcloud compute networks describe pg-lab-vpc >/dev/null 2>&1 || \
  gcloud compute networks create pg-lab-vpc --subnet-mode=custom --bgp-routing-mode=regional

gcloud compute networks subnets describe pg-lab-subnet --region=asia-south1 >/dev/null 2>&1 || \
  gcloud compute networks subnets create pg-lab-subnet \
    --network=pg-lab-vpc --region=asia-south1 --range=10.20.0.0/24 --enable-private-ip-google-access

gcloud compute networks describe pg-lab-vpc --format='yaml(name,autoCreateSubnetworks)'\
gcloud compute networks subnets describe pg-lab-subnet --region=asia-south1 --format='yaml(name,ipCidrRange,privateIpGoogleAccess)'
```

Step 4 — Networking 1.2 firewalls

```
gcloud compute firewall-rules describe pg-lab-allow-iap-ssh >/dev/null 2>&1 || \
  gcloud compute firewall-rules create pg-lab-allow-iap-ssh \
    --network=pg-lab-vpc --allow=tcp:22 --source-ranges=35.235.240.0/20 --target-tags=pg-lab

gcloud compute firewall-rules describe pg-lab-allow-5432-internal >/dev/null 2>&1 || \
  gcloud compute firewall-rules create pg-lab-allow-5432-internal \
    --network=pg-lab-vpc --allow=tcp:5432 --source-ranges=10.20.0.0/24 --target-tags=pg-lab
```

```
gcloud compute firewall-rules list --filter="network=pg-lab-vpc"
```

Step 5 — Networking 1.3 PSA for Cloud SQL

```
gcloud compute addresses describe pg-lab-psa --global >/dev/null 2>&1 || \
  gcloud compute addresses create pg-lab-psa --global --purpose=VPC_PEERING \
    --prefix-length=16 --network=pg-lab-vpc
```

```
gcloud services vpc-peerings list --network=pg-lab-vpc
```

```
gcloud services vpc-peerings connect --service=servicenetworking.googleapis.com \
  --ranges=pg-lab-psa --network=pg-lab-vpc || true
```

```
gcloud compute addresses describe pg-lab-psa --global
gcloud services vpc-peerings list --network=pg-lab-vpc
```

|| true on connect is OK if peering already exists. Do not create a second PSA address.

Step 6 — Terraform files (versions + variables + root)

```
cd "$HOME/tf-ansible-pg"
```

```
cat > "$HOME/tf-ansible-pg/versions.tf" <<'EOF'
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
  }
}
provider "google" {
  project = var.project_id
  region  = var.region
}
EOF
```

```
cat > "$HOME/tf-ansible-pg/variables.tf" <<'EOF'
variable "project_id" { type = string }
variable "region" { type = string }
variable "zone" { type = string }
variable "network_name" { type = string }
variable "subnet_name" { type = string }
variable "subnet_cidr" { type = string }
variable "vm_name" { type = string }
variable "sql_name" { type = string }
variable "sql_password" { type = string; sensitive = true }
variable "create_network" { type = bool; default = false }
EOF
```

Step 7 — Network module file

```
cat > "$HOME/tf-ansible-pg/modules/network/main.tf" <<'EOF'
variable "create_network" { type = bool }
variable "network_name" { type = string }
variable "subnet_name" { type = string }
variable "subnet_cidr" { type = string }
variable "region" { type = string }

data "google_compute_network" "this" { name = var.network_name }
data "google_compute_subnetwork" "this" {
  name    = var.subnet_name
  region  = var.region
}
resource "google_compute_global_address" "psa" {
  name          = "pg-lab-psa"
  purpose       = "VPC_PEERING"
  address_type  = "INTERNAL"
  prefix_length = 16
  network       = data.google_compute_network.this.id
}
resource "google_service_networking_connection" "psa" {
  network           = data.google_compute_network.this.id
  service           = "servicenetworking.googleapis.com"
  reserved_peering_ranges = [google_compute_global_address.psa.name]
}
resource "google_compute_firewall" "iap_ssh" {
  name          = "pg-lab-allow-iap-ssh"
  network       = data.google_compute_network.this.name
  allow { protocol = "tcp"; ports = ["22"] }
  source_ranges = ["35.235.240.0/20"]
  target_tags   = ["pg-lab"]
}
resource "google_compute_firewall" "pg" {
  name = "pg-lab-allow-5432-internal"
```

```

network = data.google_compute_network.this.name
allow { protocol = "tcp"; ports = ["5432"] }
source_ranges = [var.subnet_cidr]
target_tags   = ["pg-lab"]
}
output "network_id" { value = data.google_compute_network.this.id }
output "subnet_self_link" { value = data.google_compute_subnetwork.this.self_link }
EOF

```

Step 8 — IAM module file

```

cat > "$HOME/tf-ansible-pg/modules/iam/main.tf" <<'EOF'
variable "project_id" { type = string }
resource "google_service_account" "vm" { account_id = "pg-vm-sa"; display_name = "PostgreSQL VM" }
resource "google_service_account" "sql" { account_id = "pg-sql-client-sa"; display_name = "Cloud SQL client" }
resource "google_project_iam_member" "vm_log" {
  project = var.project_id
  role    = "roles/logging.logWriter"
  member  = "serviceAccount:${google_service_account.vm.email}"
}
resource "google_project_iam_member" "vm_mon" {
  project = var.project_id
  role    = "roles/monitoring.metricWriter"
  member  = "serviceAccount:${google_service_account.vm.email}"
}
resource "google_project_iam_member" "sql_client" {
  project = var.project_id
  role    = "roles/cloudsql.client"
  member  = "serviceAccount:${google_service_account.sql.email}"
}
output "vm_email" { value = google_service_account.vm.email }
output "sql_email" { value = google_service_account.sql.email }
EOF

```

Step 9 — Compute Engine module file

```

cat > "$HOME/tf-ansible-pg/modules/compute/main.tf" <<'EOF'
variable "name" { type = string }
variable "zone" { type = string }
variable "subnet" { type = string }
variable "sa_email" { type = string }
data "google_compute_image" "ubuntu" {
  family = "ubuntu-2404-lts-amd64"
  project = "ubuntu-os-cloud"
}
resource "google_compute_instance" "vm" {
  name         = var.name
  machine_type = "e2-medium"
  zone        = var.zone
  tags        = ["pg-lab"]
  boot_disk {
    initialize_params {
      image = data.google_compute_image.ubuntu.self_link
      size  = 20
      type  = "pd-balanced"
    }
  }
  network_interface { subnetwork = var.subnet }
  service_account {
    email = var.sa_email
    scopes = ["cloud-platform"]
  }
  metadata = { enable-oslogin = "TRUE" }
}
output "internal_ip" { value = google_compute_instance.vm.network_interface[0].network_ip }
EOF

```

Step 10 — Cloud SQL module file

```

cat > "$HOME/tf-ansible-pg/modules/sql/main.tf" <<'EOF'
variable "name" { type = string }
variable "region" { type = string }
variable "network_id" { type = string }
variable "password" { type = string; sensitive = true }
resource "google_sql_database_instance" "this" {
  name         = var.name
  database_version = "POSTGRES_16"
  region      = var.region
  settings {
    tier = "db-custom-1-3840"
    ip_configuration {
      ipv4_enabled = false
      private_network = var.network_id
    }
  }
}

```

```

    }
    availability_type = "ZONAL"
    disk_size        = 20
  }
  root_password = var.password
}
resource "google_sql_database" "appdb" {
  name       = "appdb"
  instance   = google_sql_database_instance.this.name
}
resource "google_sql_user" "appuser" {
  name       = "appuser"
  instance   = google_sql_database_instance.this.name
  password   = var.password
}
output "connection_name" { value = google_sql_database_instance.this.connection_name }
output "private_ip" { value = google_sql_database_instance.this.private_ip_address }
EOF

```

Step 11 — Root main.tf

```

cat > "$HOME/tf-ansible-pg/main.tf" <<'EOF'
module "network" {
  source          = "./modules/network"
  create_network = var.create_network
  network_name    = var.network_name
  subnet_name     = var.subnet_name
  subnet_cidr     = var.subnet_cidr
  region          = var.region
}
module "iam" {
  source      = "./modules/iam"
  project_id = var.project_id
}
module "compute" {
  source      = "./modules/compute"
  name        = var.vm_name
  zone        = var.zone
  subnet      = module.network.subnet_self_link
  sa_email    = module.iam.vm_email
}
module "sql" {
  source      = "./modules/sql"
  name        = var.sql_name
  region      = var.region
  network_id  = module.network.network_id
  password    = var.sql_password
  depends_on = [module.network]
}
output "vm_ip" { value = module.compute.internal_ip }
output "sql_connection" { value = module.sql.connection_name }
output "sql_private_ip" { value = module.sql.private_ip }
EOF
ls -R "$HOME/tf-ansible-pg"

```

Step 12 — terraform init, import existing 1.1–1.3 objects, plan, apply

```

cd "$HOME/tf-ansible-pg"
terraform init

```

Import only if plan would CREATE pg-lab-psa / firewalls that already exist. Replace YOUR_PROJECT with the same id as in tfvars.

```

cd "$HOME/tf-ansible-pg"
PROJECT_ID="$(gcloud config get-value project)"

terraform import -var-file="$HOME/tf-ansible-pg/env/terraform.tfvars" \
  module.network.google_compute_global_address.psa pg-lab-psa || true

terraform import -var-file="$HOME/tf-ansible-pg/env/terraform.tfvars" \
  'module.network.google_service_networking_connection.psa' \
  "projects/${PROJECT_ID}/global/networks/pg-lab-vpc:servicenetworking.googleapis.com" || true

terraform import -var-file="$HOME/tf-ansible-pg/env/terraform.tfvars" \
  module.network.google_compute_firewall.iap_ssh pg-lab-allow-iap-ssh || true

terraform import -var-file="$HOME/tf-ansible-pg/env/terraform.tfvars" \
  module.network.google_compute_firewall.pg pg-lab-allow-5432-internal || true

terraform plan -var-file="$HOME/tf-ansible-pg/env/terraform.tfvars" -out=tfplan
terraform apply tfplan
terraform output

```

Step 13 — Ansible playbook on the VM

```

cd "$HOME/tf-ansible-pg"
VM_IP="$(terraform output -raw vm_ip)"
echo "VM_IP=$VM_IP"

cat > "$HOME/tf-ansible-pg/ansible/inventories/lab.ini" <<EOF
[postgres]
pg-ansible-vm ansible_host=${VM_IP} ansible_user=$(whoami) ansible_python_interpreter=/usr/bin/python3
EOF

cat > "$HOME/tf-ansible-pg/ansible/playbooks/postgres.yml" <<'EOF'
---
- hosts: postgres
  become: true
  vars:
    pg_version: 16
  tasks:
    - apt:
        name:
          - "postgresql-{{ pg_version }}"
          - "postgresql-contrib-{{ pg_version }}"
          - python3-psycopg2
        update_cache: true
    - lineinfile:
        path: "/etc/postgresql/{{ pg_version }}/main/postgresql.conf"
        regexp: "^#?listen_addresses"
        line: "listen_addresses = '*'"
        notify: reload postgres
    - lineinfile:
        path: "/etc/postgresql/{{ pg_version }}/main/pg_hba.conf"
        line: "host all all 10.20.0.0/24 scram-sha-256"
        notify: reload postgres
    - file:
        path: /var/backups/postgresql
        state: directory
        owner: postgres
        mode: "0750"
    - become_user: postgres
      community.postgresql.postgresql_user:
        name: appuser
        password: "ChangeMe-Lab-Only"
        role_attr_flags: LOGIN
    - become_user: postgres
      community.postgresql.postgresql_db:
        name: appdb
        owner: appuser
  handlers:
    - name: reload postgres
      service:
        name: postgresql
        state: reloaded
EOF

```

Run Ansible ON the VM if IAP from this laptop is awkward. Copy the playbook then use local connection:

```
gcloud compute ssh pg-ansible-vm --zone=asia-south1-a --tunnel-through-iap
```

```

# inside the VM:
sudo apt-get update
sudo apt-get install -y ansible python3-psycopg2
ansible-galaxy collection install community.postgresql
# copy postgres.yml onto the VM first, then:
ansible-playbook -i 'localhost,' -c local postgres.yml

```

Step 14 — Verify

```

gcloud compute ssh pg-ansible-vm --zone=asia-south1-a --tunnel-through-iap --command='
sudo -u postgres psql -c "SELECT version();"
sudo -u postgres pg_dump -Fc appdb -f /var/backups/postgresql/appdb.dump
ls -l /var/backups/postgresql/
'

gcloud sql instances describe pg-lab-sql --format='yaml(name,ipAddresses,settings.ipConfiguration)'
cd "$HOME/tf-ansible-pg" && terraform output

```

Step 15 — Cleanup

```

cd "$HOME/tf-ansible-pg"
terraform destroy -var-file="$HOME/tf-ansible-pg/env/terraform.tfvars"

```

If you see: env/terraform.tfvars: No such file or directory

Cause: env/ was not created, or you were in \$HOME not \$HOME/tf-ansible-pg. Fix: run Step 0 then Step 1 again using the full path \$HOME/tf-ansible-pg/env/terraform.tfvars.